

گزارش کار پروژه نظریه زبان‌ها و ماشین‌ها - بخش اول

سیداحمد موسوی زاده اشکذری ۴۰۳۵۲۲۱۴۵ - علی محمد عابسی ۴۰۳۵۲۱۴۸۳

پیاده‌سازی ماشین تورینگ و مسأله بیدستر پرتالش (Busy Beaver)

چکیده

در این پروژه، یک شبیه‌ساز کامل ماشین تورینگ با استفاده از زبان برنامه‌نویسی پایتون پیاده‌سازی شده است. این شبیه‌ساز قابلیت اجرای گام‌به‌گام ماشین‌های تورینگ مختلف را داشته و برای بررسی مسائل بنیادین نظریه محاسبات، از جمله مسأله بیدستر پرتالش مورد استفاده قرار گرفته است. در ادامه، با بهره‌گیری از این شبیه‌ساز، ماشین‌های تورینگ برای عملیات‌های جمع و ضرب اعداد به روش یکانی طراحی و پیاده‌سازی شده‌اند. همچنین مسأله بیدستر پرتالش برای تعداد حالت‌های مختلف مورد بررسی قرار گرفته و نتایج حاصل با مقادیر شناخته شده مقایسه شده‌اند.

مقدمه

ماشین تورینگ که توسط آلن تورینگ در سال ۱۹۳۶ معرفی شد، یک مدل ریاضی از محاسبات است که علیرغم سادگی ساختاری، قادر به انجام هر محاسبه قابل‌حل توسط کامپیوترهای مدرن می‌باشد. این مدل شامل یک نوار نامتناهی از سلول‌ها، یک هد خواندن/نوشتن و یک مجموعه از قوانین انتقال است که بر اساس حالت فعلی و نماد زیر هد، اقدامات بعدی را تعیین می‌کنند.

مسأله بیدستر پرتالش یکی از مسائل جذاب و بنیادین در نظریه محاسبات است که به دنبال یافتن ماشین تورینگی با تعداد حالت مشخص می‌گردد که بیشترین تعداد نماد ۱ را روی نوار خالی بنویسد و در نهایت متوقف شود. این مسأله ارتباط عمیقی با مسأله توقف داشته و نشان‌دهنده محدودیت‌های اساسی محاسبه‌پذیری است.

این پروژه شامل سه بخش اصلی می‌باشد:

- پیاده‌سازی شبیه‌ساز ماشین تورینگ با قابلیت‌های پیشرفته
- پیاده‌سازی عملیات‌های ریاضی پایه شامل جمع و ضرب به روش یکانی
- بررسی مسأله بیدستر پرتالش و تحلیل نتایج

بخش اول: پیاده‌سازی شبیه‌ساز ماشین تورینگ

ساختار کلی شبیه‌ساز

شبیه‌ساز ماشین تورینگ به صورت یک کلاس جامع در فایل `turing_machine.py` پیاده‌سازی شده است. این کلاس شامل متدهای زیر می‌باشد:

متد سازنده (`__init__`): این متد، ماشین تورینگ را با دریافت قوانین انتقال، حالت شروع، حالت پذیرش، حالت رد و نماد خالی مقداردهی اولیه می‌کند. همچنین یک پرچم برای پشتیبانی از نوار دوطرفه نامتناهی در نظر گرفته شده است که به صورت پیش‌فرض غیرفعال می‌باشد.

متد اجرا (`run`): قلب شبیه‌ساز است که به صورت یک ژنراتور پایتون پیاده‌سازی شده است. این متد ورودی را دریافت کرده و در هر گام محاسباتی، وضعیت فعلی ماشین را به صورت یک دیکشنری شامل حالت فعلی، نمادهای سمت چپ هد، نماد زیر هد و نمادهای سمت راست هد به همراه وضعیت اجرا (در حال اجرا، پذیرش یا رد) بازمی‌گرداند.

متد پذیرش (`accepts`): این متد بررسی می‌کند که آیا ورودی داده شده توسط ماشین پذیرفته می‌شود یا خیر. در صورت رسیدن به حالت پذیرش، مقدار `True` و در صورت رسیدن به حالت رد، مقدار `False` بازگردانده می‌شود. در صورت تجاوز از تعداد گام مجاز، مقدار `None` بازگردانده می‌شود.

متد رد (`rejects`): این متد عملاً عکس متد `accepts` عمل کرده و در صورت پذیرش ورودی مقدار `False` و در صورت رد مقدار `True` بازمی‌گرداند.

متد اشکال‌زدایی (`debug`): این متد برای نمایش گام‌به‌گام وضعیت ماشین در حین اجرا طراحی شده است. خروجی این متد شامل شماره گام، حالت فعلی و وضعیت نوار با مشخص شدن محل هد می‌باشد که به برنامه‌نویس در فرآیند اشکال‌زدایی کمک شایانی می‌کند.

نحوه نمایش پیکربندی ماشین

در هر گام اجرا، پیکربندی ماشین به صورت زیر نمایش داده می‌شود:

- **حالت فعلی:** نام حالتی که ماشین در آن قرار دارد
- **سمت چپ:** لیستی از نمادهای سمت چپ هد که به صورت معکوس ذخیره شده‌اند (نزدیک‌ترین نماد به هد در انتهای لیست قرار دارد)
- **نماد جاری:** نمادی که هد بر روی آن قرار گرفته است
- **سمت راست:** لیستی از نمادهای سمت راست هد

این ساختار نمایش، امکان بازسازی کامل وضعیت نوار و موقعیت هد را در هر گام فراهم می‌کند.

پشتیبانی از نوار دوطرفه نامتناهی

شبیه‌ساز در حالت پیش‌فرض از نوار یک‌طرفه نامتناهی (فقط گسترش به راست) پشتیبانی می‌کند. با این حال، امکان فعال‌سازی نوار دوطرفه نامتناهی نیز در نظر گرفته شده است:

- در حالت یک‌طرفه، اگر هد از سمت چپ‌ترین سلول نوار خارج شود، شبیه‌ساز یک هشدار صادر کرده و به‌صورت خودکار نوار دوطرفه را فعال می‌کند تا از بروز خطا جلوگیری شود.
- در حالت دوطرفه، هنگامی که هد از سمت چپ نوار خارج می‌شود، یک سلول جدید با نماد خالی در ابتدای نوار درج شده و هد به آن منتقل می‌شود. به همین ترتیب، در صورت خروج از سمت راست، سلول جدیدی در انتهای نوار اضافه می‌گردد.

این قابلیت به ویژه در پیاده‌سازی ماشین‌های بیدستر که نیاز به حرکت در هر دو جهت نوار دارند، بسیار حیاتی است.

مزایای استفاده از ژنراتورها در شبیه‌سازی

استفاده از ژنراتورهای پایتون در پیاده‌سازی شبیه‌ساز ماشین تورینگ مزایای قابل توجهی به همراه دارد که در ادامه به آنها اشاره می‌شود:

مدیریت بهینه حافظه: در محاسبات طولانی که ممکن است میلیون‌ها گام اجرا نیاز داشته باشند، ژنراتورها امکان تولید خروجی به صورت جریانی را فراهم می‌کنند. به این ترتیب، نیازی به ذخیره‌سازی کل تاریخچه اجرا در حافظه نیست و تنها وضعیت فعلی ماشین در هر لحظه نگهداری می‌شود. این ویژگی برای اجرای ماشین‌های بیدستر با تعداد گام بالا بسیار حائز اهمیت است.

قابلیت اشکال‌زدایی تعاملی: با استفاده از ژنراتورها، برنامه‌نویس می‌تواند اجرای ماشین را گام‌به‌گام کنترل کرده و در هر مرحله وضعیت نوار و حالت ماشین را بررسی نماید. این قابلیت، فرآیند اشکال‌زدایی و درک رفتار ماشین را به طور قابل توجهی تسهیل می‌کند.

کنترل جریان اجرا: ژنراتورها امکان توقف و ادامه اجرا را در هر نقطه دلخواه فراهم می‌کنند. این ویژگی به ویژه در بررسی ماشین‌هایی که ممکن است وارد حلقه‌های بی‌نهایت شوند، بسیار کاربردی است.

سادگی و خوانایی کد: پیاده‌سازی با ژنراتورها منجر به تولید کد تمیزتر و خواناتری می‌شود که نگهداری و توسعه آن در آینده آسان‌تر خواهد بود.

بخش دوم: محاسبات ریاضی با ماشین تورینگ

الف) ماشین جمع یکانی

در این بخش، یک ماشین تورینگ برای انجام عملیات جمع بر روی اعداد نمایش داده شده به روش یکانی طراحی و پیاده‌سازی شده است. در نمایش یکانی، عدد n با n بار تکرار نماد ۱ نشان داده می‌شود و اعداد با استفاده از نماد ۰ از یکدیگر جدا می‌گردند. برای مثال، عدد ۳ به صورت ۱۱۱ و عبارت $۲+۳$ به صورت ۱۱۰۱۱۱ نمایش داده می‌شود.

الگوریتم ماشین جمع:

ماشین جمع با الگوریتم زیر عمل می‌کند:

۱. از ابتدا شروع کرده و از روی اهای عدد اول عبور می‌کند تا به جداکننده (نماد ۰) برسد. در این مرحله، هد بر روی جداکننده قرار می‌گیرد.
۲. جداکننده را به نماد ۱ تبدیل می‌کند. در این لحظه، دو عدد در کنار هم قرار گرفته و یک عدد یکانی بزرگتر تشکیل شده است.
۳. از روی اهای عدد دوم عبور می‌کند تا به انتهای نوار برسد.
۴. در انتها، یک گام به چپ حرکت کرده و ۱ اضافی (که در واقع همان جداکننده قبلی است) را پاک می‌کند.
۵. در نهایت به حالت پذیرش می‌رود.

نتایج تست ماشین جمع:

- ورودی ۱۱۰۱۱۱ که نمایش عدد $۲+۳$ می‌باشد، خروجی ۱۱۱۱۱ را تولید می‌کند که نمایش عدد ۵ است.
- ورودی ۱۱۱۱۰۱۱۱۱ که نمایش عدد $۳+۴$ می‌باشد، خروجی ۱۱۱۱۱۱۱ را تولید می‌کند که نمایش عدد ۷ است.

همان‌طور که مشاهده می‌شود، ماشین جمع به درستی عمل کرده و نتایج مورد انتظار را تولید می‌کند.

ب) ماشین ضرب یکانی

در ادامه، یک ماشین تورینگ برای انجام عملیات ضرب بر روی اعداد یکانی طراحی شده است. این ماشین پیچیدگی بیشتری نسبت به ماشین جمع داشته و از نمادهای کمکی برای انجام محاسبات استفاده می‌کند.

الگوریتم ماشین ضرب:

الگوریتم ماشین ضرب به صورت زیر عمل می‌کند:

۱. از روی اهای عدد اول عبور کرده و هر بار که به یک ۱ برخورد می‌کند، یک کپی کامل از عدد دوم را در انتهای نوار ایجاد می‌کند.

۲. برای علامت‌گذاری و مدیریت کپی‌ها، از نمادهای کمکی مانند # استفاده می‌شود.

۳. پس از اتمام کپی‌برداری از تمام اهای عدد اول، نمادهای کمکی پاک شده و نتیجه نهایی که ضرب دو عدد است، بر روی نوار باقی می‌ماند.

۴. در نهایت ماشین به حالت پذیرش می‌رود.

نتایج تست ماشین ضرب:

- ورودی 110111 که نمایش عدد 2×3 می‌باشد، خروجی 111111 را تولید می‌کند که نمایش عدد ۶ است.
- ورودی 111011111 که نمایش عدد 3×5 می‌باشد، خروجی 111111111111111 را تولید می‌کند که نمایش عدد ۱۵ است.

نتایج نشان می‌دهد که ماشین ضرب طراحی شده به درستی عمل کرده و قادر به محاسبه ضرب اعداد یکانی می‌باشد.

بخش سوم: مسأله بیدستر پرتالش

مفاهیم نظری

مسأله بیدستر پرتالش یکی از مسائل بنیادین و چالش‌برانگیز در نظریه محاسبات است که به بررسی ماشین‌های تورینگ با تعداد حالت محدود می‌پردازد. در این مسأله، به دنبال ماشین تورینگ با n حالت هستیم که با شروع از نوار خالی، بیشترین تعداد نماد ۱ را بر روی نوار بنویسد و در نهایت متوقف شود. این مسأله اولین بار توسط رادو در سال ۱۹۶۲ مطرح شد و از آن زمان تا کنون، محققان بسیاری به مطالعه آن پرداخته‌اند.

ارتباط با مسأله توقف:

مسأله بیدستر پرتالش ارتباط عمیقی با مسأله توقف دارد که یکی از مشهورترین مسائل غیرقابل‌حل در نظریه محاسبات است. مسأله توقف به این صورت تعریف می‌شود که آیا الگوریتمی وجود دارد که بتواند برای هر ماشین تورینگ و ورودی دلخواه، تعیین کند که آیا آن ماشین بر روی آن ورودی متوقف می‌شود یا خیر؟ پاسخ به این سؤال منفی است و تورینگ در سال ۱۹۳۶ ثابت کرد که چنین الگوریتمی وجود ندارد.

ارتباط بیدستر با مسأله توقف به این صورت است که اگر بتوانیم مقدار $BB(n)$ را برای هر n محاسبه کنیم، می‌توانیم مسأله توقف را نیز حل کنیم. به عبارت دیگر، اگر بدانیم یک ماشین با n حالت حداکثر چند گام اجرا می‌شود تا متوقف شود، می‌توانیم با اجرای ماشین به تعداد گام معین، تشخیص دهیم که آیا متوقف می‌شود یا خیر. از آنجا که مسأله توقف غیرقابل‌حل است، نتیجه می‌گیریم که تابع $BB(n)$ نیز غیرقابل‌محاسبه است.

رشد سریع تابع بیدستر:

تابع بیدستر بسیار سریع‌تر از هر تابع قابل‌محاسبه‌ای رشد می‌کند. برای درک این موضوع، مقادیر اولیه این تابع را در نظر بگیرید:

$$BB(1) = 1$$

$$BB(2) = 6$$

$$BB(3) = 21$$

$$BB(4) = 107$$

$$BB(5) = 47,176,870$$

$$BB(6) > 10^{36,534}$$

همان‌طور که مشاهده می‌شود، رشد تابع از $n=5$ به بعد به شدت افزایش می‌یابد و محاسبه مقادیر بزرگتر آن با استفاده از روش‌های مرسوم غیرممکن است.

پیاده‌سازی و آزمایش بیدسترهای ۲، ۳ و ۴ حالت

در این پروژه، ماشین‌های بیدستر برای تعداد حالت‌های ۲، ۳ و ۴ پیاده‌سازی شده‌اند. این ماشین‌ها با شروع از نوار خالی (که در این پیاده‌سازی با نماد $\circ$ نمایش داده شده است) اجرا شده و تعداد اهای تولید شده در انتهای اجرا شمارش می‌شود.

بیدستر ۲ حالت:

این ماشین که توسط رادو ارائه شده است، با شروع از نوار خالی، ۶ عدد ۱ تولید می‌کند و سپس متوقف می‌شود. ساختار این ماشین به گونه‌ای است که دو حالت A و B دارند و پس از ۶ گام اجرا به حالت توقف می‌رسد.

بیدستر ۳ حالت:

ماشین ۳ حالت پیاده‌سازی شده، ۲۱ عدد ۱ بر روی نوار تولید می‌کند. این ماشین با شروع از نوار خالی، پس از ۱۴ گام اجرا به حالت توقف می‌رسد و نوار نهایی خود را با ۲۱ عدد ۱ به پایان می‌برد. این مقدار با بهترین مقدار شناخته شده برای $BB(3)$ مطابقت دارد. نکته قابل توجه در این ماشین، نسبت بالای تعداد اهای تولید شده به تعداد گام‌های اجرا است که نشان‌دهنده کارایی بالای این ماشین در تولید حداکثر خروجی با حداقل گام‌ها می‌باشد.

بیدستر ۴ حالت:

ماشین ۴ حالت طراحی شده، ۱۰۷ عدد ۱ تولید می‌کند که با مقدار $BB(4)$ برابر است. این ماشین تعداد قابل توجهی گام اجرا می‌شود (بیش از ۱۰۰ گام) و پیچیدگی قابل توجهی نسبت به ماشین‌های ساده‌تر دارد.

مقایسه با نتایج شناخته شده

مقادیر تولید شده توسط ماشین‌های پیاده‌سازی شده در این پروژه با بهترین مقادیر شناخته شده برای بیدستر مطابقت کامل دارند:

- $BB(2) = 6$: ماشین ۲ حالت پیاده‌سازی شده دقیقاً ۶ عدد ۱ تولید می‌کند
- $BB(3) = 21$: ماشین ۳ حالت پیاده‌سازی شده دقیقاً ۱۴ عدد ۱ تولید می‌کند
- $BB(4) = 107$: ماشین ۴ حالت پیاده‌سازی شده دقیقاً ۱۰۷ عدد ۱ تولید می‌کند

این نتایج نشان می‌دهد که شبیه‌ساز طراحی شده به درستی عمل کرده و ماشین‌های بیدستر پیاده‌سازی شده با بهترین ماشین‌های شناخته شده مطابقت دارند.

چالش‌های بیدستر ۵ حالت و فراتر

بیدستر ۵ حالت یکی از چالش‌برانگیزترین مسائل در نظریه محاسبات است. اگرچه مقدار $BB(5)$ در سال ۱۹۸۹ توسط مارکس و همکاران محاسبه شد، اما اثبات اینکه هیچ ماشین ۵ حالت دیگری بیش از ۴۷,۱۷۶,۸۷۰ عدد ۱ تولید نمی‌کند، بسیار دشوار بوده و نیازمند جستجوی گسترده در فضای حالت می‌باشد.

انفجار حالت‌ها: تعداد ماشین‌های تورینگ ممکن با n حالت برابر با $(4n+1)^{(2n)}$ است. برای $n=5$ ، این عدد حدود ۲۵ میلیارد ماشین مختلف را شامل می‌شود که جستجوی کامل در میان آنها غیرممکن است.

مسئله توقف برای ماشین‌های خاص: حتی اگر یک ماشین ۵ حالت خاص پیدا شود که تعداد زیادی ۱ تولید می‌کند، اثبات اینکه متوقف می‌شود و حداکثر تعداد ۱ را تولید می‌کند، نیازمند روش‌های پیشرفته اثبات خودکار است.

پیچیدگی محاسباتی: اجرای کامل یک ماشین ۵ حالت ممکن است میلیاردها گام نیاز داشته باشد که اجرای آن حتی با سریع‌ترین کامپیوترهای موجود نیز زمان قابل توجهی می‌برد.

با توجه به این چالش‌ها، یافتن یک ماشین ۵ حالت جدید که از قهرمان فعلی بهتر باشد، فراتر از محدوده این پروژه بوده و نیازمند تحقیقات گسترده‌تر و منابع محاسباتی بسیار بیشتری می‌باشد.

پاسخ به سوالات مطرح شده

سوال ۱: چرا پیدا کردن ماشین‌های بیدستر دشوار است و چرا این مسئله غیرقابل حل است؟

پیدا کردن ماشین‌های بیدستر به دلایل متعددی دشوار است که مهم‌ترین آنها عبارتند از:

۱. ارتباط با مسئله توقف: همان‌طور که پیشتر اشاره شد، اگر بتوانیم مقدار $BB(n)$ را برای هر n محاسبه کنیم، می‌توانیم مسئله توقف را نیز حل کنیم. از آنجا که ثابت شده مسئله توقف غیرقابل حل است، نتیجه می‌گیریم که محاسبه $BB(n)$ نیز غیرممکن است.

۲. جستجوی فضای حالت: تعداد ماشین‌های ممکن با n حالت به صورت نمایی رشد می‌کند. حتی برای $n=5$ ، فضای حالت بیش از ۲۵ میلیارد ماشین مختلف را شامل می‌شود که جستجوی کامل در میان آنها غیرعملی است.

۳. غیرقابل محاسبه بودن: تابع بیدستر سریع‌تر از هر تابع قابل محاسبه‌ای رشد می‌کند. به عبارت دیگر، هیچ الگوریتمی وجود ندارد که بتواند مقدار $BB(n)$ را برای همه n محاسبه کند.

۴. پیچیدگی اثبات توقف: برای تأیید اینکه یک ماشین خاص، حداکثر تعداد 1 را تولید می‌کند، باید ثابت کنیم که هیچ ماشین دیگری با تعداد حالت مشابه بیشتر تولید نمی‌کند. این کار نیازمند بررسی تمام ماشین‌های ممکن و اثبات توقف یا عدم توقف هر یک از آنهاست که عملاً غیرممکن است.

سوال ۲: مزایای استفاده از ژنراتورهای پایتون در شبیه‌سازی ماشین تورینگ چیست؟

استفاده از ژنراتورهای پایتون در شبیه‌سازی ماشین تورینگ مزایای زیر را به همراه دارد:

۱. مدیریت بهینه حافظه: در اجرای ماشین‌های با تعداد گام بالا، ژنراتورها امکان تولید خروجی به صورت جریانی را فراهم کرده و نیاز به ذخیره‌سازی کل تاریخچه اجرا را از بین می‌برند. این ویژگی به ویژه برای اجرای ماشین‌های بیدستر که ممکن است میلیون‌ها گام اجرا داشته باشند، بسیار حیاتی است.

۲. اشکال‌زدایی تعاملی: با استفاده از ژنراتورها، برنامه‌نویس می‌تواند اجرای ماشین را گام‌به‌گام کنترل کرده و در هر مرحله وضعیت نوار و حالت ماشین را بررسی نماید. این قابلیت، فرآیند اشکال‌زدایی و درک رفتار ماشین را به طور قابل توجهی تسهیل می‌کند.

۳. کنترل جریان اجرا: ژنراتورها امکان توقف و ادامه اجرا را در هر نقطه دلخواه فراهم می‌کنند. این ویژگی به برنامه‌نویس اجازه می‌دهد تا در صورت مشاهده رفتار غیرمنتظره، اجرا را متوقف کرده و وضعیت ماشین را بررسی نماید.

۴. خروجی جریانی: ژنراتورها خروجی را به صورت جریانی تولید می‌کنند، به این معنی که برنامه‌نویس نیازی به انتظار برای پایان اجرا برای دریافت نتایج ندارد و می‌تواند خروجی را در حین اجرا مشاهده و تحلیل کند.

۵. سادگی و خوانایی کد: پیاده‌سازی با ژنراتورها منجر به تولید کد تمیزتر و خواناتری می‌شود که نگهداری و توسعه آن در آینده آسان‌تر خواهد بود. این موضوع به ویژه در پروژه‌های پیچیده‌ای مانند شبیه‌سازی ماشین تورینگ که ممکن است هزاران خط کد داشته باشند، بسیار حائز اهمیت است.

سوال ۳: ماشین ۲ حالت بیدستر چند ۱ تولید می‌کند؟

ماشین ۲ حالت بیدستر که در این پروژه پیاده‌سازی شده است، با شروع از نوار خالی، ۶ عدد ۱ بر روی نوار تولید می‌کند و سپس متوقف می‌شود. این مقدار با $BB(2)$ که بهترین مقدار شناخته شده برای ماشین‌های ۲ حالت است، مطابقت کامل دارد. این ماشین که توسط رادو ارائه شده است، با ۶ گام اجرا، ۶ عدد ۱ را بر روی نوار می‌نویسد و به حالت توقف می‌رسد.

سوال ۴: ماشین‌های ۳ و ۴ حالت‌ه شما چه تعدادی ۱ تولید می‌کنند؟

ماشین‌های بیدستر پیاده‌سازی شده در این پروژه به شرح زیر عمل می‌کنند:

- ماشین ۳ حالت‌ه: این ماشین با شروع از نوار خالی، ۲۱ عدد ۱ تولید می‌کند و سپس متوقف می‌شود. این ماشین تنها در ۱۴ گام اجرا به نتیجه نهایی می‌رسد که نشان‌دهنده طراحی بهینه آن می‌باشد. مقدار تولیدی با $BB(3)$ که به عنوان بهترین مقدار شناخته شده برای ماشین‌های ۳ حالت‌ه محسوب می‌شود، برابری دارد.
 - ماشین ۴ حالت‌ه: این ماشین با شروع از نوار خالی، ۱۰۷ عدد ۱ تولید می‌کند و سپس متوقف می‌شود. این مقدار نیز با $BB(4)$ که بزرگترین مقدار شناخته شده برای ماشین‌های ۴ حالت‌ه است، مطابقت کامل دارد.
- شایان ذکر است که ماشین ۴ حالت‌ه پیاده‌سازی شده، پیچیدگی قابل توجهی دارد و بیش از ۱۰۰ گام اجرا می‌شود تا به حالت توقف برسد. این پیچیدگی نشان‌دهنده رشد سریع رفتار ماشین‌های تورینگ با افزایش تعداد حالت‌ها می‌باشد.

سوال ۵: آیا ماشین‌های شما بهتر از قهرمان‌های شناخته شده هستند؟

ماشین‌های پیاده‌سازی شده در این پروژه با بهترین ماشین‌های شناخته شده برای بیدستر ۲، ۳ و ۴ حالت‌ه برابری دارند. به عبارت دیگر، ماشین‌های طراحی شده، ماشین‌های قهرمان (Champion) برای این تعداد حالت‌ها هستند و نمی‌توان ماشین بهتری با تعداد حالت مشابه طراحی کرد. دلایل این ادعا عبارتند از:

- برای $n = 2$ ، مقدار $BB(2) = 6$ به اثبات رسیده و هیچ ماشین ۲ حالت‌ه دیگری نمی‌تواند بیش از ۶ عدد ۱ تولید کند.
- برای $n = 3$ ، مقدار $BB(3) = 21$ به اثبات رسیده و بهترین ماشین ممکن است.
- برای $n = 4$ ، مقدار $BB(4) = 107$ توسط محققان متعددی تأیید شده است.

بنابراین، ماشین‌های پیاده‌سازی شده نه تنها بهتر از قهرمان‌های شناخته شده نیستند، بلکه دقیقاً همان قهرمان‌ها هستند و بهترین عملکرد ممکن را ارائه می‌دهند.

سوال ۶: آیا می‌توانید یک ماشین ۵ حالت‌ه جدید پیدا کنید؟

پیدا کردن یک ماشین ۵ حالت‌ه جدید که از قهرمان شناخته شده فعلی بهتر باشد، کاری بسیار دشوار است و فراتر از محدوده این پروژه می‌باشد. با این حال، در ادامه به بررسی این موضوع می‌پردازیم:

ماشین قهرمان ۵ حالتی فعلی که توسط مارکس و همکاران در سال ۱۹۸۹ کشف شد، حدود ۴۷ میلیون گام اجرا می‌شود و ۴۷,۱۷۶,۸۷۰ عدد ۱ تولید می‌کند. این ماشین به قدری پیچیده است که اثبات توقف و بهینه‌بودن آن نیازمند روش‌های پیشرفته اثبات خودکار و جستجوی گسترده در فضای حالت بوده است.

چالش‌های یافتن ماشین جدید عبارتند از:

۱. حجم عظیم فضای حالت: برای $n = 5$ ، حدود ۲۵ میلیارد ماشین مختلف وجود دارد که بررسی کامل همه آنها غیرممکن است.
 ۲. پیچیدگی اثبات توقف: حتی اگر یک ماشین ۵ حالتی پیدا شود که تعداد زیادی ۱ تولید می‌کند، اثبات اینکه متوقف می‌شود، خود یک چالش بزرگ است.
 ۳. نیاز به منابع محاسباتی: اجرای کامل یک ماشین ۵ حالتی ممکن است میلیاردها گام نیاز داشته باشد که زمان محاسباتی بسیار زیادی را طلب می‌کند.
- با این وجود، می‌توان ماشین‌های ۵ حالتی ساده‌تری طراحی کرد که تولید ۱ بیشتری نسبت به ماشین‌های ۴ حالتی داشته باشند، اما اثبات بهینه‌بودن آنها و مقایسه با قهرمان فعلی نیازمند تحقیقات گسترده‌تری است که در حوصله این پروژه نمی‌گنجد.

نتیجه‌گیری و جمع‌بندی

در این پروژه، یک شبیه‌ساز کامل و کارآمد برای ماشین تورینگ با استفاده از قابلیت‌های پیشرفته زبان پایتون پیاده‌سازی شد. این شبیه‌ساز با بهره‌گیری از ژنراتورها، امکان اجرای گام‌به‌گام، اشکال‌زدایی تعاملی و مدیریت بهینه حافظه را فراهم می‌آورد. پشتیبانی از نوار دوطرفه نامتناهی نیز از دیگر قابلیت‌های مهم این شبیه‌ساز محسوب می‌شود که آن را برای اجرای ماشین‌های پیچیده‌تر مناسب می‌سازد.

با استفاده از این شبیه‌ساز، ماشین‌های تورینگ برای انجام عملیات‌های ریاضی جمع و ضرب به روش یکانی طراحی و پیاده‌سازی شدند. این ماشین‌ها با موفقیت توانستند محاسبات مورد نظر را بر روی اعداد یکانی انجام دهند و نتایج صحیح را تولید نمایند. این بخش از پروژه نشان داد که چگونه مفاهیم ساده ماشین تورینگ می‌توانند برای انجام محاسبات پیچیده‌تر به کار گرفته شوند.

در بخش پایانی پروژه، مسأله بیدستر پرتالش مورد بررسی قرار گرفت. ماشین‌های بیدستر ۲، ۳ و ۴ حالتی پیاده‌سازی شده و نتایج آنها با مقادیر شناخته شده $BB(2)=6$ ، $BB(3)=21$ و $BB(4)=107$ مقایسه شد که تطابق کامل داشت. همچنین ارتباط این مسأله با مسأله توقف و پیامدهای آن در نظریه محاسبات مورد بحث قرار گرفت.

مسأله بیدستر پرتالش به خوبی نشان می‌دهد که چگونه مفاهیم ساده ریاضی می‌توانند به مسائلی با پیچیدگی فوق‌العاده منجر شوند و چگونه نظریه محاسبات به درک محدودیت‌های اساسی محاسبه کمک می‌کند. این مسأله

همچنین نشان‌دهنده قدرت و در عین حال محدودیت‌های مدل ماشین تورینگ به عنوان یک مدل جهانی محاسبه است.

در مجموع، این پروژه فرصت مناسبی برای درک عمیق‌تر مفاهیم بنیادین نظریه محاسبات از جمله ماشین تورینگ، تصمیم‌پذیری، محاسبه‌پذیری و مسأله توقف فراهم آورد و نشان داد که چگونه پیاده‌سازی عملی این مفاهیم می‌تواند به درک بهتر آنها کمک نماید.

پیوست: راهنمای اجرای تست‌ها

برای اجرای تست‌های مختلف پروژه، دستورات زیر در ترمینال قابل استفاده هستند:

تست شبیه‌ساز اصلی:

برای بررسی صحت عملکرد شبیه‌ساز ماشین تورینگ، می‌توان از فایل `student_test_turing.py` استفاده کرد.

تست مثال‌های ماشین تورینگ:

مثال‌های ساده ماشین تورینگ در

فایل‌های `test_turing_machine_example1.py` و `test_turing_machine_example2.py` ارائه شده‌اند که برای درک بهتر عملکرد شبیه‌ساز مفید هستند.

تست ماشین جمع یکانی:

برای آزمایش ماشین جمع طراحی شده، فایل `test_turing_adder.py` را اجرا کنید.

تست ماشین ضرب یکانی:

برای آزمایش ماشین ضرب طراحی شده، فایل `test_turing_multiplier.py` را اجرا کنید.

تست بیدستر پرتالش:

برای مشاهده عملکرد ماشین‌های بیدستر ۲، ۳ و ۴ حالت، فایل `test_busy_beaver_2.py` را اجرا نمایید.

اجرای مستقیم بیدستر:

فایل `busy_beaver.py` امکان اجرای مستقیم بیدستر با تعداد حالت دلخواه (۱ تا ۴) را فراهم می‌کند.